Day 20: CQRS, Read-Model Projections & Eventual Consistency

Separating write and read paths so systems can scale and evolve independently.

CQRS Basics

- Separate command and query responsibilities
- Write model enforces business rules
- Read model serves optimized queries

Command Side

- Validate intent
- Update write model
- Emit domain events

Query Side

- Read-optimized views
- Denormalized projections
- Fast query responses

Eventual Consistency

- Read model can lag writes
- Stale reads are temporary
- APIs should expose processing state

CQRS Projection Flow

1. Client sends command
2. Command Service validates request
3. Write Store saves aggregate state
4. Event Log appends event
5. Projector consumes new events
6. Read Store updates projection
7. Query Service returns read model

Replay & Rebuild

- Reprocess event log when needed
- Rebuild read model safely
- Idempotent projectors are critical

Trade-Offs

- Better scalability for complex domains
- More moving parts and monitoring
- Projection lag must be managed

CQRS improves scalability by optimizing writes and reads independently, while projection pipelines keep query models updated asynchronously.

Day 20: CQRS, Read-Model Projections, and Eventual Consistency

Day 20 focuses on separating write-side command handling from read-side query models so systems can scale and evolve independently.

It covers:

- command-query responsibility segregation (CQRS) fundamentals
- write model vs read model responsibilities
- event log based projection pipeline
- eventual consistency between command and query paths
- replay and read-model rebuild behavior
- design trade-offs for complexity vs scalability

In short, Day 20 is about optimizing writes and reads separately while handling asynchronous projection lag safely.

Core ideas

Why CQRS exists

- Write workflows and read workflows usually have different performance and schema needs.
- A single model optimized for both often becomes slow and hard to maintain.
- CQRS separates command handling from query read paths.

Command side

- Command APIs validate intent and enforce business rules.
- Successful commands mutate the write model and emit events.
- Command responses focus on acknowledgement, not rich query payloads.

Query side

- Read model is built for query patterns, not transaction invariants.
- Projectors consume events and update denormalized views.
- Query APIs stay fast because they avoid write-side logic.

Eventual consistency

- Read model can lag behind recent writes.
- Clients may temporarily observe stale data until projection catches up.
- UIs and APIs should communicate processing state where needed.

Trade-offs

- CQRS improves scalability and clarity for complex domains.
- It increases architecture complexity (projectors, offsets, replay, monitoring).
- Idempotent projection and replay-safe handlers are critical for correctness.

Day-20 sample: CQRS Order Projection

This repository includes a small CQRS sample in both Python and Java.

Functional requirements

- Accept commands to create and confirm orders
- Store command-side aggregate state
- Emit domain events to an append-only event log
- Build a query-side read model using a projector
- Demonstrate lag between write and read paths

High-level components

- CommandService: validates and applies write-side commands
- WriteStore: aggregate state and event log
- Projector: applies events to read model with tracked offset
- ReadStore and QueryService: query-side optimized access

Data flow

1. Client sends command (create_order, confirm_order)
2. Command side validates and writes aggregate state
3. Command side appends event to event log
4. Projector polls new events from last offset
5. Read model is updated from projected events
6. Query API serves denormalized read model

Project structure

```
Day-20/
  README.md
  cqrs-sample/
    python/
      cqrs_order_system.py
      demo.py
    java/
      CqrsOrderSystem.java
      Main.java
```

Run the sample

Python

```
cd cqrs-sample\python
python demo.py
```

Java

```
cd cqrs-sample\java
javac Main.java CqrsOrderSystem.java
java Main
```

Interview takeaways

- CQRS is useful when read/write workloads and models diverge significantly
- Event projection lag is normal and should be explicit in design discussion
- Replay capability is essential for read-model recovery and evolution
- Idempotent projectors prevent corruption during retries or reprocessing
- CQRS can pair effectively with Outbox, Saga, and event-driven systems

Next improvements

- Add projection retries and dead-letter handling
- Add idempotency keys for command deduplication
- Add snapshotting for faster aggregate rebuilds
- Add multiple read models for different query shapes
- Add metrics for projection lag and event throughput

Code Appendix

Python and Java source used in Day 20 sample.

cqrs-sample\python\cqrs_order_system.py

```
from __future__ import annotations

from dataclasses import dataclass
from enum import Enum


class OrderStatus(Enum):
    CREATED = "CREATED"
    CONFIRMED = "CONFIRMED"


@dataclass(frozen=True)
class OrderAggregate:
    order_id: str
    customer_id: str
    amount_cents: int
    status: OrderStatus


@dataclass(frozen=True)
class DomainEvent:
    sequence: int
    event_type: str
    order_id: str
    customer_id: str
    amount_cents: int


@dataclass
class OrderReadModel:
    order_id: str
    customer_id: str
    amount_cents: int
    status: str


class WriteStore:
    def __init__(self) -> None:
        self.orders: dict[str, OrderAggregate] = {}
        self.events: list[DomainEvent] = []
        self._next_sequence = 1

    def append_event(
        self, event_type: str, order_id: str, customer_id: str, amount_cents: int
    ) -> DomainEvent:
        event = DomainEvent(
            sequence=self._next_sequence,
            event_type=event_type,
            order_id=order_id,
            customer_id=customer_id,
            amount_cents=amount_cents,
        )
        self._next_sequence += 1
        self.events.append(event)
        return event


class ReadStore:
    def __init__(self) -> None:
        self.orders: dict[str, OrderReadModel] = {}


class CommandService:
    def __init__(self, write_store: WriteStore) -> None:
```

```
        self._write_store = write_store

    def create_order(self, order_id: str, customer_id: str, amount_cents: int) -> None:
        if order_id in self._write_store.orders:
            raise ValueError(f"order already exists: {order_id}")
        if amount_cents <= 0:
            raise ValueError("amount_cents must be positive")

        aggregate = OrderAggregate(
            order_id=order_id,
            customer_id=customer_id,
            amount_cents=amount_cents,
            status=OrderStatus.CREATED,
        )
        self._write_store.orders[order_id] = aggregate
        self._write_store.append_event(
            event_type="OrderCreated",
            order_id=order_id,
            customer_id=customer_id,
            amount_cents=amount_cents,
        )

    def confirm_order(self, order_id: str) -> None:
        aggregate = self._write_store.orders.get(order_id)
        if aggregate is None:
            raise ValueError(f"order not found: {order_id}")
        if aggregate.status == OrderStatus.CONFIRMED:
            return

        confirmed = OrderAggregate(
            order_id=aggregate.order_id,
            customer_id=aggregate.customer_id,
            amount_cents=aggregate.amount_cents,
            status=OrderStatus.CONFIRMED,
        )
        self._write_store.orders[order_id] = confirmed
        self._write_store.append_event(
            event_type="OrderConfirmed",
            order_id=order_id,
            customer_id=aggregate.customer_id,
            amount_cents=aggregate.amount_cents,
        )


class Projector:
    def __init__(self, write_store: WriteStore, read_store: ReadStore) -> None:
        self._write_store = write_store
        self._read_store = read_store
        self._last_sequence = 0

    @property
    def last_sequence(self) -> int:
        return self._last_sequence

    def project_next(self, batch_size: int = 10) -> list[str]:
        history: list[str] = []
        if batch_size <= 0:
            return history

        unapplied = [e for e in self._write_store.events if e.sequence > self._last_sequence]
        for event in unapplied[:batch_size]:
            if event.event_type == "OrderCreated":
                self._read_store.orders[event.order_id] = OrderReadModel(
                    order_id=event.order_id,
                    customer_id=event.customer_id,
                    amount_cents=event.amount_cents,
                    status=OrderStatus.CREATED.value,
                )
                history.append(f"projected {event.event_type}#{event.sequence}")
            elif event.event_type == "OrderConfirmed":
                row = self._read_store.orders.get(event.order_id)
```

```
                if row is not None:
                    row.status = OrderStatus.CONFIRMED.value
                history.append(f"projected {event.event_type}#{event.sequence}")
            else:
                history.append(f"skipped unknown event#{event.sequence}")
            self._last_sequence = event.sequence

        return history


class QueryService:
    def __init__(self, read_store: ReadStore) -> None:
        self._read_store = read_store

    def get_order(self, order_id: str) -> OrderReadModel | None:
        row = self._read_store.orders.get(order_id)
        if row is None:
            return None
        return OrderReadModel(
            order_id=row.order_id,
            customer_id=row.customer_id,
            amount_cents=row.amount_cents,
            status=row.status,
        )

    def list_orders_for_customer(self, customer_id: str) -> list[OrderReadModel]:
        rows = [r for r in self._read_store.orders.values() if r.customer_id == customer_id]
        return [
            OrderReadModel(
                order_id=r.order_id,
                customer_id=r.customer_id,
                amount_cents=r.amount_cents,
                status=r.status,
            )
            for r in rows
        ]
```

cqrs-sample\python\demo.py

```
from cqrs_order_system import CommandService, Projector, QueryService, ReadStore, WriteStore


def print_order(prefix: str, order) -> None:
    if order is None:
        print(prefix, "<not available in read model>")
        return
    print(
        prefix,
        f"id={order.order_id} customer={order.customer_id} amount={order.amount_cents} status={order.status
    )


def main() -> None:
    write_store = WriteStore()
    read_store = ReadStore()

    commands = CommandService(write_store)
    projector = Projector(write_store, read_store)
    queries = QueryService(read_store)

    commands.create_order("order-200", "cust-1", 4999)
    print_order("before projection:", queries.get_order("order-200"))

    for line in projector.project_next(batch_size=10):
        print(line)
    print_order("after create projection:", queries.get_order("order-200"))

    commands.confirm_order("order-200")
    print_order("after confirm, before projection:", queries.get_order("order-200"))

    for line in projector.project_next(batch_size=10):
```

```
        print(line)
    print_order("after confirm projection:", queries.get_order("order-200"))

    print("projector last_sequence:", projector.last_sequence)
    print("event count:", len(write_store.events))


if __name__ == "__main__":
    main()
```

cqrs-sample\java\CqrsOrderSystem.java

```
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

public class CqrsOrderSystem {
    public enum OrderStatus {
        CREATED,
        CONFIRMED
    }

    public static final class OrderAggregate {
        private final String orderId;
        private final String customerId;
        private final int amountCents;
        private final OrderStatus status;

        public OrderAggregate(String orderId, String customerId, int amountCents, OrderStatus status) {
            this.orderId = orderId;
            this.customerId = customerId;
            this.amountCents = amountCents;
            this.status = status;
        }

        public String getOrderId() {
            return orderId;
        }

        public String getCustomerId() {
            return customerId;
        }

        public int getAmountCents() {
            return amountCents;
        }

        public OrderStatus getStatus() {
            return status;
        }
    }

    public static final class DomainEvent {
        private final int sequence;
        private final String eventType;
        private final String orderId;
        private final String customerId;
        private final int amountCents;

        public DomainEvent(
            int sequence,
            String eventType,
            String orderId,
            String customerId,
            int amountCents
        ) {
            this.sequence = sequence;
            this.eventType = eventType;
            this.orderId = orderId;
            this.customerId = customerId;
```

```
        this.amountCents = amountCents;
    }

    public int getSequence() {
        return sequence;
    }

    public String getEventType() {
        return eventType;
    }

    public String getOrderId() {
        return orderId;
    }

    public String getCustomerId() {
        return customerId;
    }

    public int getAmountCents() {
        return amountCents;
    }
}

public static final class OrderReadModel {
    private final String orderId;
    private final String customerId;
    private final int amountCents;
    private String status;

    public OrderReadModel(String orderId, String customerId, int amountCents, String status) {
        this.orderId = orderId;
        this.customerId = customerId;
        this.amountCents = amountCents;
        this.status = status;
    }

    public String getOrderId() {
        return orderId;
    }

    public String getCustomerId() {
        return customerId;
    }

    public int getAmountCents() {
        return amountCents;
    }

    public String getStatus() {
        return status;
    }

    public void setStatus(String status) {
        this.status = status;
    }
}

public static final class WriteStore {
    private final Map<String, OrderAggregate> orders = new LinkedHashMap<>();
    private final List<DomainEvent> events = new ArrayList<>();
    private int nextSequence = 1;

    public DomainEvent appendEvent(
        String eventType,
        String orderId,
        String customerId,
        int amountCents
    ) {
        DomainEvent event = new DomainEvent(
            nextSequence,
```

```
            eventType,
            orderId,
            customerId,
            amountCents
        );
        nextSequence += 1;
        events.add(event);
        return event;
    }

    public Map<String, OrderAggregate> getOrders() {
        return orders;
    }

    public List<DomainEvent> getEvents() {
        return events;
    }
}

public static final class ReadStore {
    private final Map<String, OrderReadModel> orders = new LinkedHashMap<>();

    public Map<String, OrderReadModel> getOrders() {
        return orders;
    }
}

public static final class CommandService {
    private final WriteStore writeStore;

    public CommandService(WriteStore writeStore) {
        this.writeStore = writeStore;
    }

    public void createOrder(String orderId, String customerId, int amountCents) {
        if (writeStore.getOrders().containsKey(orderId)) {
            throw new IllegalArgumentException("order already exists: " + orderId);
        }
        if (amountCents <= 0) {
            throw new IllegalArgumentException("amountCents must be positive");
        }

        OrderAggregate aggregate = new OrderAggregate(
            orderId,
            customerId,
            amountCents,
            OrderStatus.CREATED
        );
        writeStore.getOrders().put(orderId, aggregate);
        writeStore.appendEvent("OrderCreated", orderId, customerId, amountCents);
    }

    public void confirmOrder(String orderId) {
        OrderAggregate aggregate = writeStore.getOrders().get(orderId);
        if (aggregate == null) {
            throw new IllegalArgumentException("order not found: " + orderId);
        }
        if (aggregate.getStatus() == OrderStatus.CONFIRMED) {
            return;
        }

        OrderAggregate confirmed = new OrderAggregate(
            aggregate.getOrderId(),
            aggregate.getCustomerId(),
            aggregate.getAmountCents(),
            OrderStatus.CONFIRMED
        );
        writeStore.getOrders().put(orderId, confirmed);
        writeStore.appendEvent(
            "OrderConfirmed",
            orderId,
```

```
            aggregate.getCustomerId(),
            aggregate.getAmountCents()
        );
    }
}

public static final class Projector {
    private final WriteStore writeStore;
    private final ReadStore readStore;
    private int lastSequence = 0;

    public Projector(WriteStore writeStore, ReadStore readStore) {
        this.writeStore = writeStore;
        this.readStore = readStore;
    }

    public int getLastSequence() {
        return lastSequence;
    }

    public List<String> projectNext(int batchSize) {
        List<String> history = new ArrayList<>();
        if (batchSize <= 0) {
            return history;
        }

        int processed = 0;
        for (DomainEvent event : writeStore.getEvents()) {
            if (event.getSequence() <= lastSequence) {
                continue;
            }
            if (processed >= batchSize) {
                break;
            }

            if ("OrderCreated".equals(event.getEventType())) {
                readStore.getOrders().put(
                    event.getOrderId(),
                    new OrderReadModel(
                        event.getOrderId(),
                        event.getCustomerId(),
                        event.getAmountCents(),
                        OrderStatus.CREATED.name()
                    )
                );
                history.add("projected " + event.getEventType() + "#" + event.getSequence());
            } else if ("OrderConfirmed".equals(event.getEventType())) {
                OrderReadModel row = readStore.getOrders().get(event.getOrderId());
                if (row != null) {
                    row.setStatus(OrderStatus.CONFIRMED.name());
                }
                history.add("projected " + event.getEventType() + "#" + event.getSequence());
            } else {
                history.add("skipped unknown event#" + event.getSequence());
            }

            lastSequence = event.getSequence();
            processed += 1;
        }

        return history;
    }
}

public static final class QueryService {
    private final ReadStore readStore;

    public QueryService(ReadStore readStore) {
        this.readStore = readStore;
    }
```

```
        public OrderReadModel getOrder(String orderId) {
            OrderReadModel row = readStore.getOrders().get(orderId);
            if (row == null) {
                return null;
            }
            return new OrderReadModel(
                row.getOrderId(),
                row.getCustomerId(),
                row.getAmountCents(),
                row.getStatus()
            );
        }

        public List<OrderReadModel> listOrdersForCustomer(String customerId) {
            List<OrderReadModel> result = new ArrayList<>();
            for (OrderReadModel row : readStore.getOrders().values()) {
                if (!customerId.equals(row.getCustomerId())) {
                    continue;
                }
                result.add(
                    new OrderReadModel(
                        row.getOrderId(),
                        row.getCustomerId(),
                        row.getAmountCents(),
                        row.getStatus()
                    )
                );
            }
            return result;
        }
    }
}
```

cqrs-sample\java\Main.java

```
public class Main {
    private static void printOrder(String prefix, CqrsOrderSystem.OrderReadModel order) {
        if (order == null) {
            System.out.println(prefix + " <not available in read model>");
            return;
        }
        System.out.println(
            prefix
                + " id="
                + order.getOrderId()
                + " customer="
                + order.getCustomerId()
                + " amount="
                + order.getAmountCents()
                + " status="
                + order.getStatus()
        );
    }

    public static void main(String[] args) {
        CqrsOrderSystem.WriteStore writeStore = new CqrsOrderSystem.WriteStore();
        CqrsOrderSystem.ReadStore readStore = new CqrsOrderSystem.ReadStore();

        CqrsOrderSystem.CommandService commands = new CqrsOrderSystem.CommandService(writeStore);
        CqrsOrderSystem.Projector projector = new CqrsOrderSystem.Projector(writeStore, readStore);
        CqrsOrderSystem.QueryService queries = new CqrsOrderSystem.QueryService(readStore);

        commands.createOrder("order-200", "cust-1", 4999);
        printOrder("before projection:", queries.getOrder("order-200"));

        for (String line : projector.projectNext(10)) {
            System.out.println(line);
        }
        printOrder("after create projection:", queries.getOrder("order-200"));

        commands.confirmOrder("order-200");
```

```
        printOrder("after confirm, before projection:", queries.getOrder("order-200"));

        for (String line : projector.projectNext(10)) {
            System.out.println(line);
        }
        printOrder("after confirm projection:", queries.getOrder("order-200"));

        System.out.println("projector last_sequence: " + projector.getLastSequence());
        System.out.println("event count: " + writeStore.getEvents().size());
    }
}
```